

Urban Flighter: A Real-Geometry, Real-Inlet Urban Drone Simulator

with Hidden Toy CFD, Quadratic Drag, and an Honest RL Contract

Jaewon Jang

Inha University

August 2026

Abstract

Urban drone routing is not a still-air waypoint problem. Buildings change the local wind, and quadratic air-relative drag changes both the trajectory a vehicle can hold and the energy it burns. Most aerial simulators optimize photorealism or vehicle dynamics; most CFD packages optimize field accuracy and are not interactive flight products. Urban Flighter sits between them. It loads real OpenStreetMap footprints for an arbitrary city, pulls a forecast-model inlet velocity from Open-Meteo, computes a cheap hidden 2D flow (potential flow with near-wall damping and an empirical wake), and flies a metre-scale quadrotor under a shared quadratic air-relative drag core. A browser cockpit exposes 2D and 3D Lite flight, deterministic LiDAR-style returns, and a simulated range-Doppler radar proxy. A headless Gymnasium-shaped environment, UrbanFlow Gym, uses the same geometry, inlet, and hidden field, but returns only a 49-dimensional actor observation: odometry, an inlet-only relative-air estimate, goal context, 16-ray geometry LiDAR, and an 8-beam radar proxy. The actor never receives the velocity grid. This paper is a system preprint, not a Navier–Stokes paper and not a trained-policy paper. Two checkable studies are reported: a still-air / headwind / tailwind integration of the drag core, and an off-line Gym fixture in which unconstrained goal-seeking collides while geometry-safe A* succeeds. An inlet-aware heuristic is *not* automatically energy-optimal. No PPO or SAC weights are shipped.

1 Introduction

A quadrotor flying between real buildings feels two things that game physics usually erases: solids that must not be hit, and air that is not still. A tailwind lowers relative airspeed; a headwind raises it; a wake behind a slab can do both in one block [9, 18]. Those effects change both the path a vehicle can hold and the energy it burns.

The practical research question is narrower than “solve urban CFD.” Given only (i) the building geometry of the place you are in and (ii) a forecast inlet velocity U_∞ , can a planner or controller choose the next route or local command so as to reduce drag and avoid collisions? That is a sensing-and-control problem, not a grid-access problem. A learning agent that sees the full hidden flow is solving a different, easier task than a vehicle that will later fly with LiDAR, a coarse radar, and a weather forecast.

Existing stacks split the wrong way. AirSim and FlightGoggles emphasize visual fidelity and vehicle dynamics [6, 15]. CARLA and PX4 SITL are mature for ground vehicles or firmware-in-the-loop flight, not urban wind-aware energy [5, 11]. Full CFD remains the right tool for pedestrian-wind studies and is the wrong tool for an interactive research cockpit [2, 16]. Urban Flighter is an open-source compromise: real geometry, real inlet, cheap hidden flow, honest sensors, and an RL contract that forbids grid leakage.

Contributions.

1. A full-stack urban-air cockpit that, for any requested city, loads OSM footprints [3, 8] and a forecast-model inlet, computes a toy CFD-lite field in interactive time, and flies a 120 Hz 2D / 3D Lite vehicle under quadratic air-relative drag.
2. A shared drag-and-power core used by the browser, a Python module, and a formula-only Unity port. Stick-off equilibrium is local wind, not still air.
3. Deterministic geometry LiDAR and a simulated range-Doppler radar proxy. Rolling maps are labelled as simulator odometry, not SLAM or RF hardware [12].
4. UrbanFlow Gym: a NumPy environment with a 49-D actor observation, leakage guards, live-OSM scenario binding, three deterministic baselines, and optional PPO/SAC extras [7, 13, 14, 17]. Status: **NOT TRAINED**.
5. Two reproducible studies whose qualitative outcomes are stated in advance.

This paper does not claim a new turbulence model, a hardware radar, or a learned energy-optimal policy. Those are future journal targets; the present artifact is the simulator and the contract.

2 Related work

AirSim couples Unreal visuals with vehicle dynamics and compares software components to real flights [15]. FlightGoggles keeps complex dynamics outside a photorealistic renderer [6]. Urban Fighter does the opposite of photogrammetry-first design: the primary object is a research contract over urban wind, drag, and observations, rendered in a browser.

Hoffmann et al. showed that even moderate airspeed changes quadrotor thrust and attitude, so hover-only models fail outdoors [9]. Ware and Roy studied wind-field estimation and exploitation in the urban canopy [18]. We take that lesson at a coarser grain: parasite drag and induced power depend on $\mathbf{v}_{\text{air}} = \mathbf{v}_g - \mathbf{w}$, not on ground speed alone [10]. The model is momentum theory plus a quadratic parasite term, not blade-element theory, and is not a claim to match STARMAC flight data.

OpenStreetMap and OSMnx supply the building layer [3, 8]. Open-Meteo supplies the 10 m forecast-model current wind used as U_∞ . The Gym-style reset/step shape follows OpenAI Gym / Gymnasium [4, 17]. Stable-Baselines3 PPO and SAC are optional extras [7, 13, 14]. The default UrbanFlow core is NumPy-only.

Commercial driving-and-drone simulators (for example MORAI-class stacks) target certification-style sensor realism. Urban Fighter is intentionally smaller: one honest research loop from real geometry and a real inlet to drag-aware flight and an RL interface.

3 System

Figure 1 is the product split. Hidden CFD-lite flow and quadratic drag drive motion and energy. The actor never receives the grid. Scenery trees, lamps, and a bundled True 3D $u/v/w$ overlay are display-only.

Backend. `/flow-fields/2d` returns OSM polygons, an Open-Meteo (or labelled fallback) inlet, and a streamfunction / potential-flow grid with near-wall damping and an empirical wake (“CFD-lite B”). A successful non-empty response is registered as an immutable snapshot `urbanflow.live_scenario.v1`.

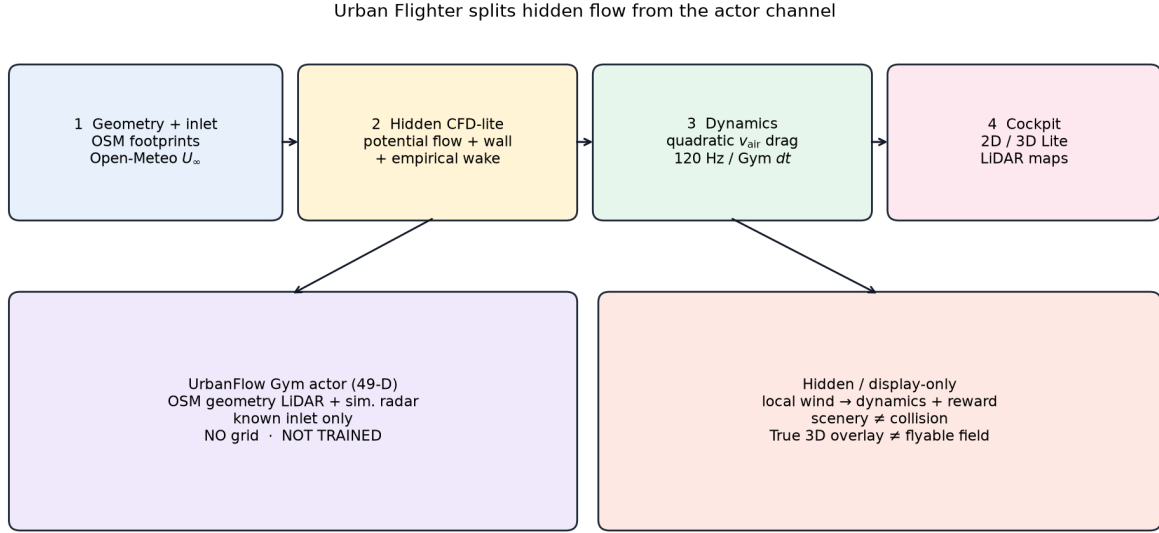


Figure 1: System split. OSM geometry and an Open-Meteo inlet feed a hidden CFD-lite grid. Flyable dynamics and UrbanFlow Gym sample that grid; the actor does not.

Frontend. Three public modes share the same flyable horizontal grid:

- **2D:** north-up canvas, 120 Hz, swept-circle clearance, 180-ray display scan.
- **3D Lite:** 0.58 m visual quad, 1.25 m horizontal / 2 m roof research envelope, building and ground returns.
- **True 3D Wind:** the same flyable city plus a bundled voxel potential-flow $u/v/w$ overlay. Overlay $\neq$ flight dynamics.

Figures 2 and 3 are chrome-hidden stills from the default Inha session, not mockups.

Honesty labels. CFD-lite $\neq$ Navier–Stokes; SIM odometry / no loop closure; radar = geometric range–Doppler proxy, not RF hardware; UrbanFlow Gym / NOT TRAINED; full flow access = NO.

4 Real geometry and real inlet

Buildings come from OSMnx `features_from_point` around a user-selected latitude and longitude [3]. Footprints are projected to a local east/north metre frame. Height uses `osm:height` if present, otherwise `building:levels` $\times 3.5$ m, otherwise a labelled 10 m default. The default demo location is Inha / Incheon (37.451448°N, 126.651542°E). Any other city is a picker click, not a code change.

The inlet is the Open-Meteo forecast-model current 10 m wind, stored as a 2D velocity (u_x, u_y) and as meteorological direction-from-north. If the fetch fails or returns an unusable vector, the payload is a deterministic 5 m/s north-origin fallback with explicit `source` and `fallback` fields. There is no silent fake sensor.

This inlet is the *known* wind: it appears on the HUD and in the actor observation. Building-scale local flow is not known to the actor.

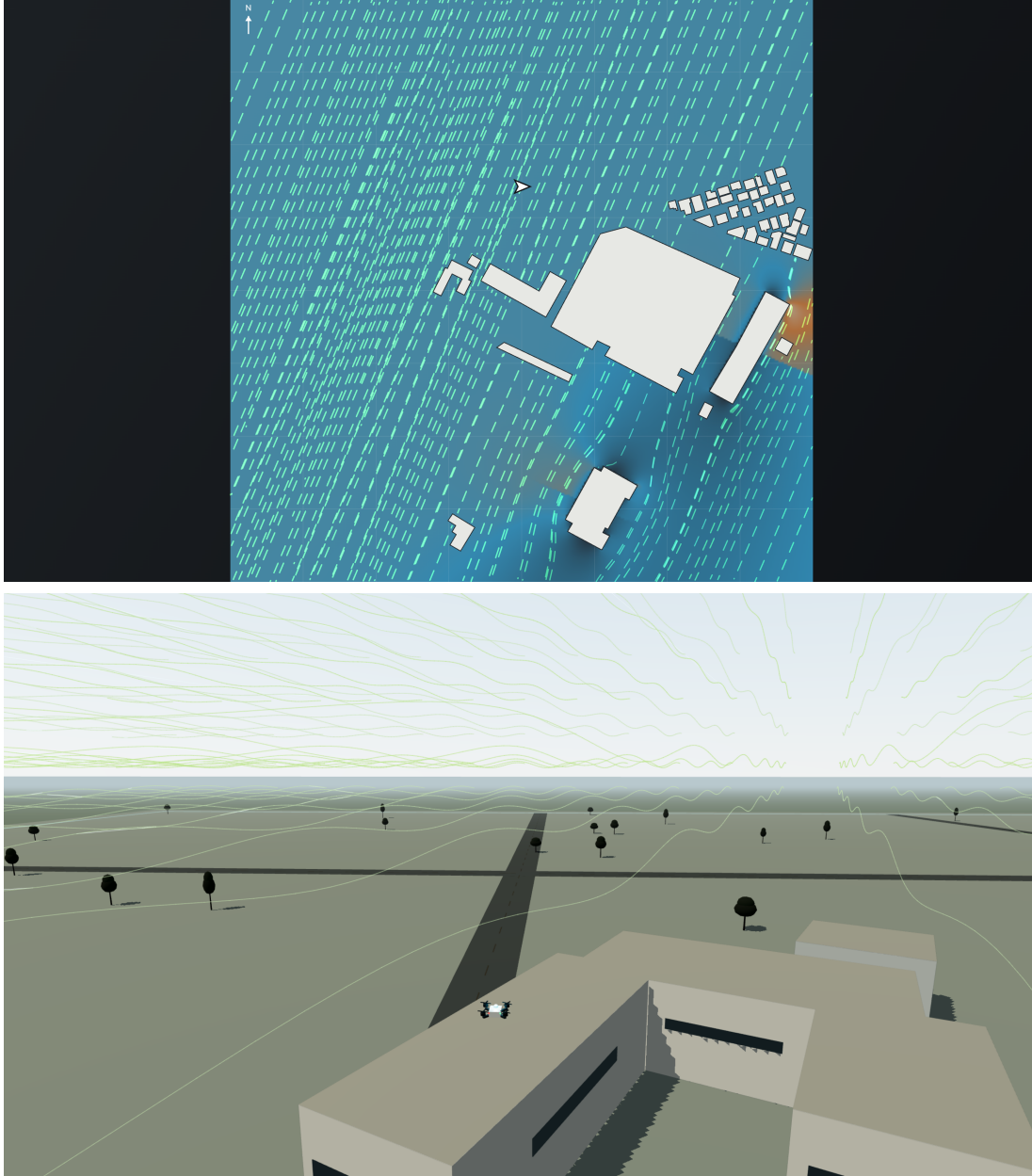


Figure 2: Live Inha / Incheon session with cockpit chrome hidden. Top: 2D north-up canvas, OSM footprints, CFD-lite colormap and streamlines that stop at the building mask. Bottom: 3D Lite with the same horizontal field; trees and roads are presentation-only. Not Navier–Stokes.

5 Hidden toy CFD

A 2D slice solves a discrete Laplace problem on a masked grid and recovers velocity from the streamfunction [1]. Inlet / far-field velocity is prescribed; building cells are solid. The production solver (CFD-lite B) then applies two cheap, named corrections that do not change the Jacobi budget:

1. **Near-wall-only damping.** Far-field cells keep the potential-flow solution; a short exponential blend slows fluid only near the mask.

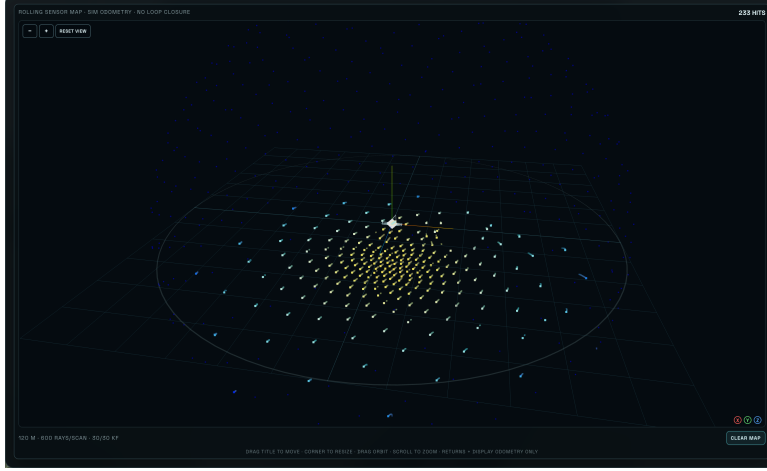


Figure 3: Live 3D rolling sensor map from the same session (233 simulator returns). Display odometry only; no loop closure, not hardware LiDAR or RF radar.

2. **Empirical wake deficit.** Downstream of each footprint, a Gaussian shelter multiplies speed. This is not vortex shedding.

Sampling is mask-aware bilinear: solid corners do not bleed into fluid interpolants. That removes the most common “streamline through a building” artifact without a finer grid.

This is cheap and produces acceleration around corners (Fig. 4). It cannot represent viscous separation, vortex shedding, or turbulence [2]. No Navier–Stokes residual is claimed. A voxel potential-flow field can supply u, v, w streamlines for visualization; the flyable aircraft still samples the live horizontal CFD-lite grid.

6 Quadratic air-relative drag

Let $\mathbf{v}_g$ be ground velocity and $\mathbf{w}$ the hidden local wind. Relative air velocity is

$$\mathbf{v}_{\text{air}} = \mathbf{v}_g - \mathbf{w}. \quad (1)$$

Parasite force and power use the standard quadratic law

$$\mathbf{F}_p = -\frac{1}{2}\rho C_d A \|\mathbf{v}_{\text{air}}\| \mathbf{v}_{\text{air}}, \quad P_p = \frac{1}{2}\rho C_d A \|\mathbf{v}_{\text{air}}\|^3. \quad (2)$$

The flyable integrators apply an implicit step that is unconditionally stable at the 120 Hz cockpit rate and at the larger Gym step:

$$\mathbf{v}_{\text{air}}^+ = \frac{\mathbf{v}_{\text{air}}}{1 + \Delta t(k\|\mathbf{v}_{\text{air}}\| + c)}, \quad k = \frac{\rho C_d A}{2m}. \quad (3)$$

With thrust off, the only equilibrium is $\mathbf{v}_g = \mathbf{w}$.

Induced power uses momentum theory, not blade element [10]:

$$v_h = \sqrt{\frac{W}{2\rho A_{\text{disk}}}}, \quad P_i = \frac{\kappa W v_h}{\sqrt{1 + (\|\mathbf{v}_{\text{air}}\|/v_h)^2}}. \quad (4)$$

Total electrical-equivalent power is avionics + sensors + $P_p + P_i$ + climb $W \max(v_z, 0)$. Default constants: $m = 2.5 \text{ kg}$, $\rho = 1.225 \text{ kg/m}^3$, $C_d = 1.05$, $A = 0.18 \text{ m}^2$, four 0.15 m propellers, $\kappa = 1.15$. Then $k = 0.0463 \text{ m}^{-1}$.

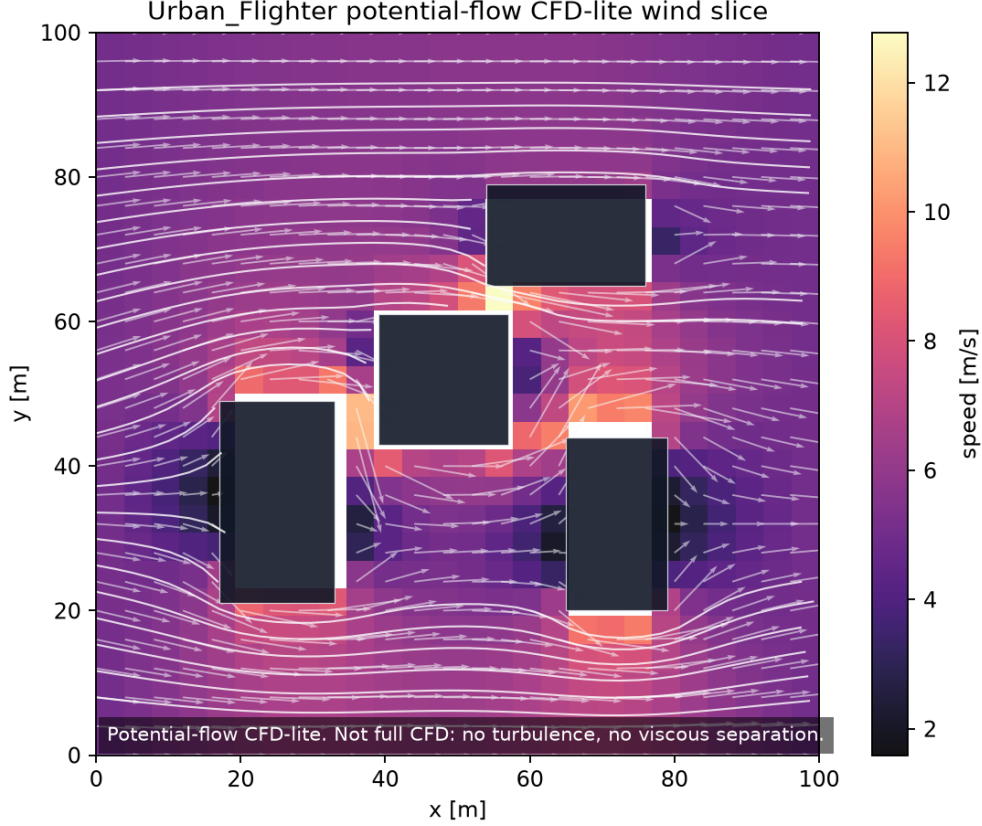


Figure 4: Offline toy-city potential-flow CFD-lite showcase. Rectangular prisms, westerly inlet, masked solids, streamlines terminated at the mask. Not Navier–Stokes.

UrbanFlow Gym now uses this same quadratic core for hidden dynamics. Reward still includes an $\|\mathbf{v}_{\text{air}}\|^2$ energy proxy so that a later trained policy can be compared against the published baselines without a silent reward rewrite. Episode metrics also record parasite energy in joules.

7 Sensing

Frontend LiDAR prototypes use a frozen five-value-per-ray layout $(d_x, d_y, d_z, \hat{r}, h)$: local direction, normalized range, and hit flag. 3D default: 600 Fibonacci-sphere samples, 120 m. 2D display default: 180 rays, 180 m. Ground is a real target: downward rays intersect $y = 0$ and take the nearer of ground and buildings.

UrbanFlow Gym uses a coarser, actor-facing sensor set (Fig. 5):

- 16-ray geometry LiDAR, 35 m, full azimuth, exact ray–polygon hits.
- 8-beam simulated radar, $\pm 60^\circ$ forward fan, 40 m. Range is the same geometric intersection. Range-rate is $-\mathbf{v}_g \cdot \hat{\mathbf{b}}$: a Doppler proxy for static buildings from own ground velocity [12].

The radar is not an FMCW waveform, has no RCS, no CFAR, and no rain model. It exists so that a later policy can use a second, narrower exteroceptive channel without inventing RF physics.

Rolling sensor maps accumulate display-only scan history with simulator odometry. They are not hardware-qualified LiDAR, loop closure, or production SLAM.

Actor observation



Hidden from reset/step

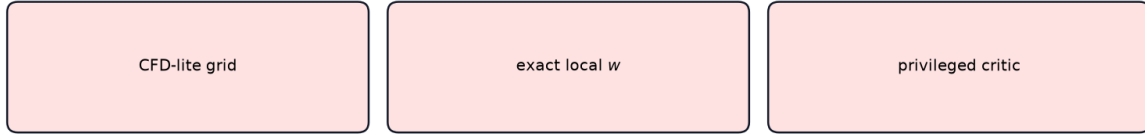


Figure 5: Actor observation versus hidden state. Relative air in the actor vector is $\mathbf{v}_g - U_\infty$, not $\mathbf{v}_g - \mathbf{w}(\mathbf{x})$.

8 UrbanFlow Gym

The core is NumPy-only. Action is a bounded 2D local-guidance command, not motor RPM. The 49-scalar actor observation is: own motion, an inlet-only relative-air estimate, known inlet, goal context, LiDAR, radar, and the previous action. Automated leakage guards reject full grids, exact wakes, future samples, and provider objects. A privileged critic helper exists and is never returned by `reset/step`.

Three deterministic baselines share only public mission context:

1. Direct goal (no obstacle planner).
2. Geometry-safe A* + waypoint tracking.
3. Inlet-aware A* with known-inlet feed-forward. Still not privileged-grid access.

Live evaluation binds to the exact UI-registered OSM scenario. Section 10 uses the synthetic rectangular fixture so the committed study needs no network.

Optional training extras install Gymnasium, Stable-Baselines3, and PyTorch [13, 17]. The training entrypoint can run PPO or SAC on the fixture or on an exported live snapshot. The evaluation entrypoint scores random, baseline, or checkpoint policies under the same observation contract. No weights are bundled. That is the foundation for a later journal paper on learned drag-minimizing control; it is not that paper.

9 Study A: drag essence

Command:

```
PYTHONPATH=backend python scripts/render_paper_figures.py
```

Still-air power split. Figure 6 integrates the analytic power model over $|\mathbf{v}_{\text{air}}| \in [0, 18]$ m/s. Hover total power is 362 W. The minimum is 350 W at 5.2 m/s. At 11 m/s the total is 427 W: induced has fallen, parasite has risen. This is the expected U-shape of a coarse momentum-plus-parasite model, not a fitted motor map.

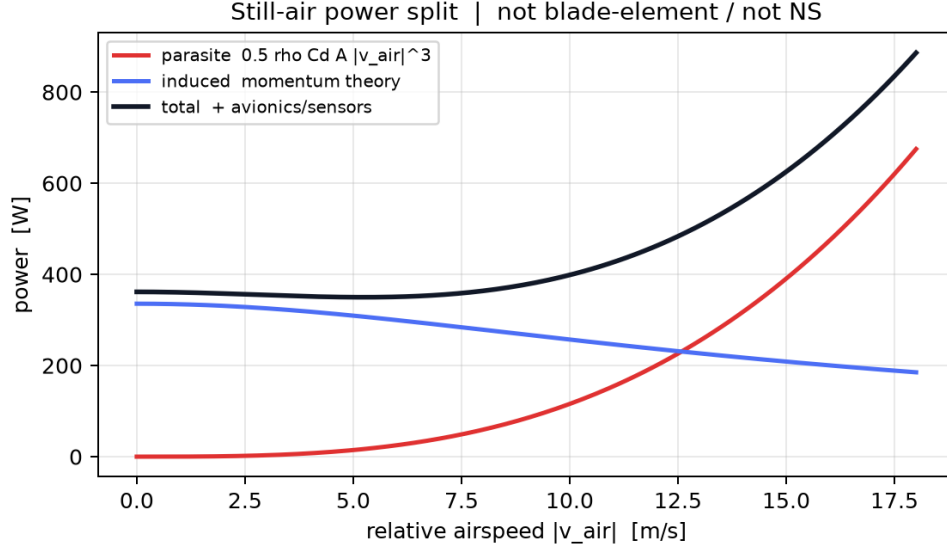


Figure 6: Still-air power versus relative airspeed. Induced falls, parasite rises. Not blade-element theory.

Same thrust, three winds. A constant 8 m/s^2 body thrust is integrated for 8 s at $\Delta t = 1/120 \text{ s}$ (Fig. 7, left). Final ground speeds: tailwind $+6 \text{ m/s} \rightarrow 16.4 \text{ m/s}$; still air $\rightarrow 10.4 \text{ m/s}$; headwind $-6 \text{ m/s} \rightarrow 4.4 \text{ m/s}$. Integrated energy: 3116 J, 3151 J, 3203 J. Headwind is slower and costlier. The energy gap is modest because the run is short and induced still dominates at these speeds; the speed gap is not.

Stick-off. Zero thrust in an 8 m/s wind reaches 7.28 m/s ground speed in 6 s (Fig. 7, right). The vehicle is drifting with the air, not parking in the world frame.

10 Study B: Gym fixture

Command (no network):

```
PYTHONPATH=backend python scripts/run_oss_showcase.py
```

Seeds {10007, 10009, 10037}, horizon 240 steps, $\Delta t = 0.25 \text{ s}$. Policy status: NOT TRAINED. Privileged flow access: false. Hidden dynamics: synthetic potential-flow-ish wake proxy plus quadratic drag, not Navier–Stokes. Evaluation id: c73930718d558dcd.

Table 1 and Figs. 8–9 are regenerated by that command. The qualitative contract is fixed in advance:

1. Direct goal collides when a slab sits on the line of sight.
2. Geometry-safe A^* reaches the goal.
3. Inlet-aware A^* is not required to beat shortest-path on energy.
4. The actor observation contains no grid.

The showcase is a contract test, not a “wind-aware always wins” claim.

Quadratic air-relative drag in the flyable loop

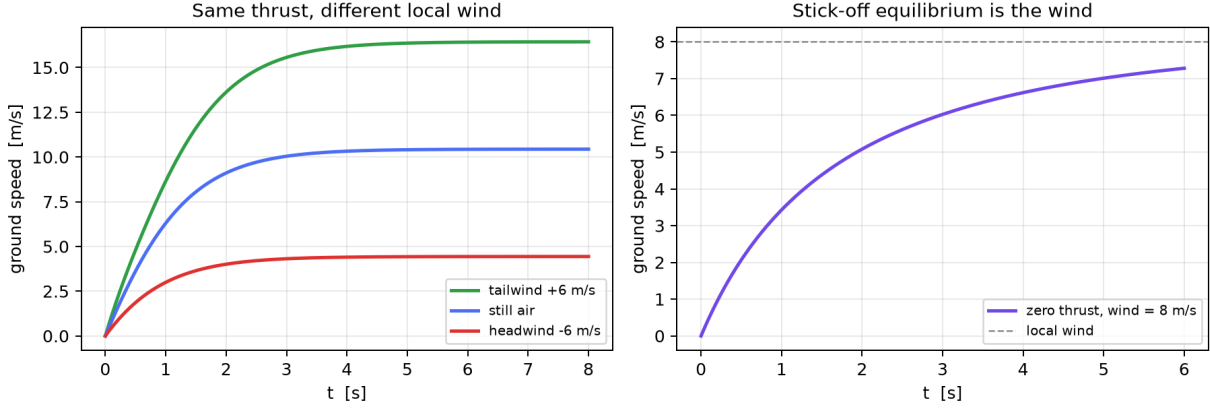


Figure 7: Left: identical thrust in tailwind, still air, and headwind. Right: thrust off; ground speed approaches the local wind.

Table 1: UrbanFlow Gym synthetic-fixture evaluation. Inlet-aware is not assumed energy-better; any regression is reported.

Baseline	Success	Mean path (m)	Rel-air energy	Collision eps.
Direct goal	0/3	23.9	158	3/3
Shortest-path A*	3/3	141.5	1099	0/3
Inlet-aware A*	3/3	141.3	1135	0/3

11 Limitations

- CFD-lite and voxel potential flow are not LES/RANS/DNS [2, 16].
- True 3D Wind is a visualization overlay over a bundled Gangnam field; it does not follow the selected city.
- Flyable motion is a damped point-mass with quadratic air drag, not rotor flapping or blade-element moments [9].
- LiDAR and rolling maps are simulator rays plus odometry, not hardware SLAM.
- Radar is a geometric range–Doppler proxy, not an electromagnetic sensor [12].
- Browser display LiDAR (180 / 600 rays) is denser than the Gym actor set (16 + 8).
- No PPO/SAC training is shipped.
- Live OSM/weather paths need network; the committed Gym showcase does not.
- Presentation geometry never enters physics.

12 Reproducibility

- Code: https://github.com/VortexyAether/Urban_Flighter

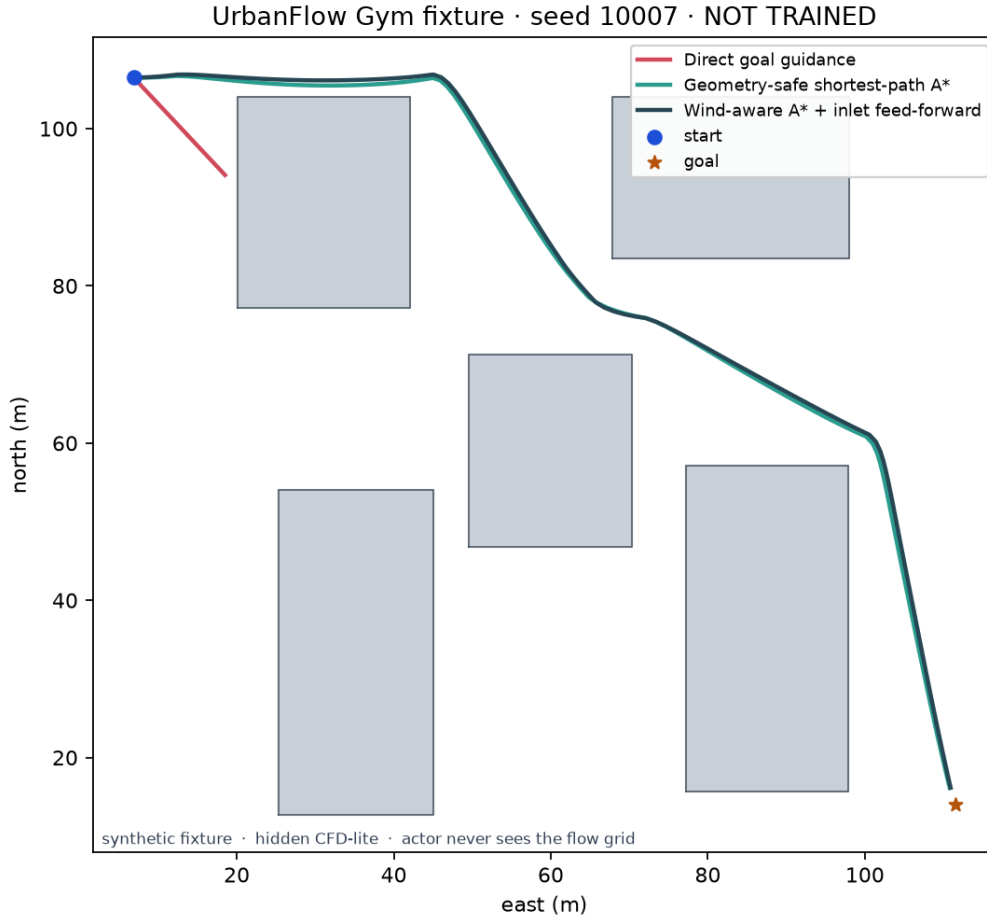


Figure 8: Seed 10007. Direct goal collides. Both planners reach the goal. Actor observation dimension is 49.

- License: MIT.
- Drag figures: Section 9.
- Gym figures and Table 1: Section 10.
- Paper figures and metrics live under `paper/figures/`.
- Live cockpit stills (Figs. 2, 3): `scripts/capture_live_shots.mjs` with backend and Vite already up.
- RL contract: `docs/urbanflow_gym_contract.md`.

13 Conclusion

Urban Flighter is an open urban-air research cockpit: real cities when requested, a real forecast inlet, cheap hidden flow, quadratic air-relative drag in the flyable loop, LiDAR and a simulated radar, and a deliberately narrow actor contract. The public numbers are intentionally small and checkable. A later paper can train

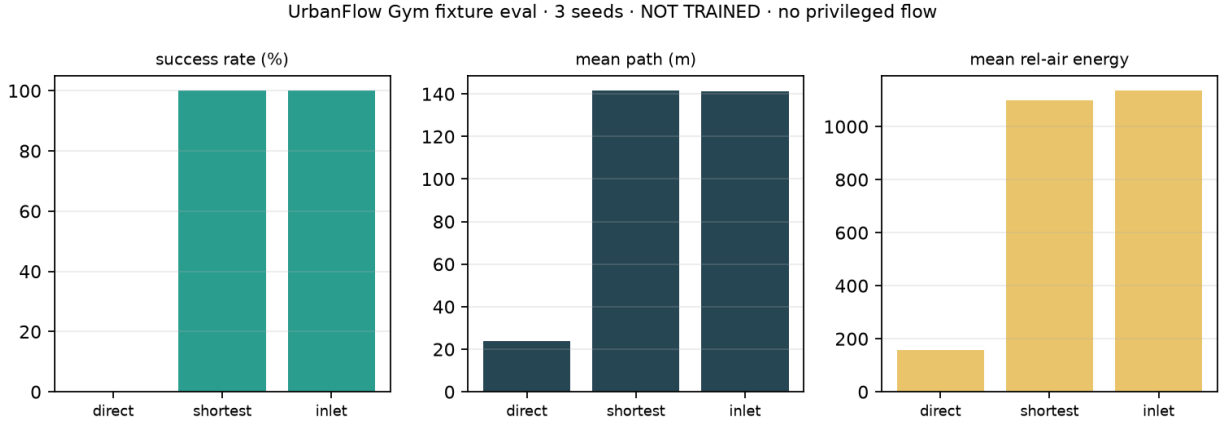


Figure 9: Aggregate fixture metrics. Geometry, not inlet heuristics, is what separates success from collision here.

PPO or SAC on that contract and ask whether geometry-plus-inlet is enough to reduce drag. This paper only builds the machine that makes that question well-posed.

References

- [1] John D. Anderson. Fundamentals of aerodynamics. *McGraw-Hill*, 2017. Book.
- [2] Bert Blocken. 50 years of computational wind engineering: Past, present and future. *Journal of Wind Engineering and Industrial Aerodynamics*, 129:69–102, 2014.
- [3] Geoff Boeing. OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, 65:126–139, 2017.
- [4] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI Gym, 2016.
- [5] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. CARLA: An open urban driving simulator. In *Proceedings of the 1st Annual Conference on Robot Learning*, 2017.
- [6] Winter Guerra, Ezra Tal, Varun Murali, Gilhyun Ryou, and Sertac Karaman. FlightGoggles: A modular framework for photorealistic camera, exteroceptive sensor, and dynamics simulation. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019. arXiv:1905.11377.
- [7] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, 2018.
- [8] Mordechai Haklay and Patrick Weber. OpenStreetMap: User-generated street maps. *IEEE Pervasive Computing*, 7(4):12–18, 2008.
- [9] Gabriel M. Hoffmann, Haomiao Huang, Steven L. Waslander, and Claire J. Tomlin. Quadrotor helicopter flight dynamics and control: Theory and experiment. In *AIAA Guidance, Navigation, and Control Conference and Exhibit*, Hilton Head, SC, 2007.

- [10] J. Gordon Leishman. Principles of helicopter aerodynamics. *Cambridge University Press*, 2006. Book.
- [11] Lorenz Meier, Dominik Honegger, and Marc Pollefeys. PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2015.
- [12] Sujeet M. Patole, Murat Torlak, Dan Wang, and Murtaza Ali. Automotive radars: A review of signal processing techniques. *IEEE Signal Processing Magazine*, 34(2):22–35, 2017.
- [13] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-Baselines3: Reliable reinforcement learning implementations. In *Journal of Machine Learning Research*, 2021.
- [14] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [15] Shital Shah, Debadeepta Dey, Chris Lovett, and Ashish Kapoor. AirSim: High-fidelity visual and physical simulation for autonomous vehicles. In *Field and Service Robotics*, 2018. arXiv:1705.05065.
- [16] Yoshihide Tominaga, Akashi Mochida, Ryuichiro Yoshie, Hiroto Kataoka, Tsuyoshi Nozu, Masaru Yoshikawa, and Taichi Shirasawa. AIJ guidelines for practical applications of CFD to pedestrian wind environment around buildings. *Journal of Wind Engineering and Industrial Aerodynamics*, 96(10–11): 1749–1761, 2008.
- [17] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deiak, Manuel Goulao, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. In *Advances in Neural Information Processing Systems*, 2024.
- [18] Justin Ware and Nicholas Roy. An analysis of wind field estimation and exploitation for quadrotor flight in the urban canopy layer. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2016.

A Software map

backend/ FastAPI and CFD-lite;
 backend/urban-flighter_physics/ shared drag;
 backend/urbanflow_gym/ headless env, train, and eval;
 frontend/src/simulation/quadraticAirDrag.ts cockpit twin;
 frontend/src/sensors/ LiDAR and sim-radar proxy;
 unity/UrbanFlighterDrag/ formula port only;
 scripts/render_paper_figures.py, scripts/run_oss_showcase.py, and
 scripts/capture_live_shots.mjs this paper.